



EAST WEST UNIVERSITY
Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Class Test-1, Spring 2026 Semester

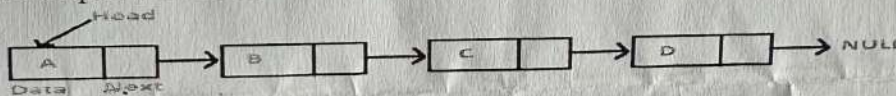
Course: CSE 207- Data Structures, Section-1
Instructor: Dr. Maheen Islam, Associate Professor, CSE Department
Full Marks: 20
Time: 40 Minutes

1. Suppose that a program has declared a structure type variable *node* and three pointer variables of *node* type.

```
struct node *head;
```

```
struct node  
{  
    char data;  
    struct node *next;  
};
```

Let, pointer *head* is a head pointer for a non-empty linked list (already created). Write some code that will return the value at the *n*'th node from end of the Linked List. For example, if we want to find *n* = 3 number node from then end of the list, then the output will be B.



2. Suppose you are given a sorted doubly linked list of positive distinct elements, the task is to find pairs in doubly linked list whose sum is equal to given value *x*. For example:

Input : head : 1 <-> 2 <-> 4 <-> 5 <-> 6 <-> 8 <-> 9
 $x = 7$

Output: (6, 1), (5, 2)

Now write an algorithm to solve the above problem.



EAST WEST UNIVERSITY
Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Mid Term Examination, Spring 2026 Semester

Course: CSE 207- Data Structures, Section-1
Instructor: Dr. Maheen Islam, Associate Professor, CSE Department
Full Marks: 30
Time: 1 Hour and 15 Minutes

Note: There are Five questions, answer ALL of them. Course Outcome (CO), Cognitive Level and Mark of each question are mentioned at the right margin.

1. Write a program that dynamically allocates memory for an array of integers. [CO1,C2,
Find the smallest value in the array and move it to the last position, keeping the relative Mark: 5]
order of other elements unchanged.

Example:

Input:

8 3 1 12 5 7

Output:

Smallest element = 1

New Array = 8 3 12 5 7 1

2. Write a program for a singly linked list that includes: [CO1,C3,
Mark: 10]
- A function to create the linked list
 - A function to rearrange the list

The rearrangement must satisfy the following conditions:

1. All nodes at odd positions remain at the beginning of the list.
2. All nodes at even positions are moved to the end of the list.
3. The relative order of nodes within both the odd-positioned group and the even-positioned group must remain unchanged.
4. The rearrangement must be performed by modifying node links (pointers) only.
Do not copy data into new nodes.
5. Positions are counted starting from 1 (i.e., the head node is at position 1).

Example:

Original List:

1 → 2 → 3 → 4 → 5 → 6

Rearranged List:

1 → 3 → 5 → 2 → 4 → 6

head → data = val;
head → null.

3. Given a doubly-linked list of integers, write a function to: [CO1,C2,
Mark: 5]
- Find the first negative value
 - Move that node to the end of the list
 - Maintain the order of remaining nodes

Example:

Input:

5 ⇌ -2 ⇌ 8 ⇌ 7 ⇌ -1

Output:

5 $\Rightarrow$ 8 $\Rightarrow$ 7 $\Rightarrow$ -1 $\Rightarrow$ -2

4. Given a stack of integers, write a program to remove consecutive duplicate elements from the stack when viewed from the top-side perspective. [CO3, C2, Mark: 5]
Two elements are considered consecutive duplicates if they appear adjacent in the stack and have the same value.
You are allowed to use one additional temporary stack, but recursion is not allowed.
After removing duplicates, the remaining elements must preserve their original relative order.

Example

Stack representation (bottom $\rightarrow$ top):

Input:

3 7 7 2 2 2 9 9 5

Output:

3 7 2 9 5

5. Given a queue of integers, write a function to rearrange the queue as follows:

[CO3, C3, Mark: 5]

1. Compute the average of all elements currently in the queue.
2. Rearrange the queue so that: all elements less than or equal to the average remain at the front of the queue and all elements greater than the average are moved to the rear of the queue.
3. While rearranging the relative order of elements in each group must remain the same as in the original queue.

You may use one additional queue if necessary.

Example:

Input (front $\rightarrow$ rear):

2 8 4 10 6

Average = $(2 + 8 + 4 + 10 + 6) / 5 = 30 / 5 = 6$

Output (front $\rightarrow$ rear):

2 4 6 8 10



EAST WEST UNIVERSITY
Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Lab Examination, Spring 2026 Semester

Course:

CSE 207- Data Structures, Section-1

Instructor:

Dr. Maheen Islam, Associate Professor, CSE Department

Full Marks:

20

Time:

1 Hour

-
1. Implement a program to represent and manipulate sparse matrices using a linked list. A sparse matrix is a matrix where most of the elements are zero. Each node in the linked list should store the row index, column index, and value of a non-zero element in the matrix.

Tasks:

1. Input the Sparse Matrix:

- Write a function to input a sparse matrix and represent it using a linked list. Each node should store:
 - Row index
 - Column index
 - Non-zero value
 - Pointer to the next node

2. Display the Sparse Matrix:

- Write a function to display the sparse matrix in its original form (i.e., as a 2D matrix). For zero elements, display 0.

3. Add Two Sparse Matrices:

- Write a function to add two sparse matrices represented as linked lists and return the resultant sparse matrix.

Sample Input and Output:

Input:

Enter the dimensions of the matrix: 4 5

Enter the number of non-zero elements: 4

Enter row, column, and value of each non-zero element:

0 1 5

1 2 8

3 0 6

3 4 9

Sparse Matrix Representation:

0 5 0 0 0

0 0 8 0 0

0 0 0 0 0

6 0 0 0 9

Enter the number of non-zero elements for another matrix (for addition): 3

Enter row, column, and value of each non-zero element:

0 1 3

1 3 4

3 0 2

Resultant Matrix after Addition:

0 8 0 0 0

0 0 8 4 0

0 0 0 0 0

8 0 0 0 9

Key Functional Requirements:

1. Data Structure for Node:

- Each node should have:
 - row (integer)
 - column (integer)
 - value (integer)
 - next (pointer to the next node)

2. Functions to Implement:

- **inputMatrix()**: Inputs non-zero elements and constructs the linked list representation of the matrix.
- **displayMatrix(matrix, rows, cols)**: Displays the sparse matrix in its original 2D form.
- **addMatrices(matrix1, matrix2)**: Adds two sparse matrices represented as linked lists.